

How to Cancel a Non-Cancellable Task in C#

In this article, I am going to discuss **How to Cancel a Non-Cancellable Task in C# using TaskCompletionSource** with Examples. Please read our previous article where we discussed [ValueTask in C#](#) with Examples.

Canceling Non-Cancellable Tasks with TaskCompletionSource in C#

We are going to see a pattern through which we can Cancel any non-cancellable task in a simple way. When we talk about non-Cancellable tasks, we mean asynchronous methods which do not receive a cancellation token, and therefore we cannot cancel using a token unless we write custom logic to achieve it. What we will do is we will use TaskCompletionSource to create a simple task that we will be able to cancel with a token. This is useful when we don't want to schedule a time out, but rather we want to have a task that does nothing, but we want to be able to cancel.

Example to understand How to Cancel Non-Cancellable Tasks in C#:

Let us see an example to understand How to Cancel Non-Cancellable Tasks in C# using TaskCompletionSource. What we are going to do is that we are going to create a new class, which is going to be a static class because we will have an extension method on it. So, I am creating a class file with the name TaskExtensionMethods.cs and then copy and paste the following code into it.

```
using System;
using System.Threading.Tasks;
using System.Threading;
namespace AsynchronousProgramming
{
    public static class TaskExtensionMethods
    {
        public static async Task<T> WithCancellation<T>(this Task<T> task,
CancellationToken cancellationToken)
        {
            var TCS = new TaskCompletionSource<object>
(TaskCreationOptions.RunContinuationsAsynchronously);
            using (cancellationToken.Register(state =>
            {
                ((TaskCompletionSource<object>)state).TrySetResult(null);
            },TCS))
            {
                var resultTask = await Task.WhenAny(task, TCS.Task);
                if(resultTask == TCS.Task)
                {
                    throw new OperationCanceledException(cancellationToken);
                }
                return await task;
            }
        }
    }
}
```

Here, we are making the class public and static as we are going to add the Extension method. The method name WithCancellation refers to the Cancellation Name of Asynchronous Stream Which we will discuss in our next article.

The first parameter of the WithCancellation method refers to the class name to which this extension method belongs. Here, it is Task<T> class. The second parameter is the normal parameter and we need to pass this parameter while calling this method and here it is a cancellation token.

As we said, we need a TaskCompletionSource. So, we need to create an instance of TaskCompletionSource.

Now, there is something called a cancellation token and it has a Register method that is going to execute a method when the token gets canceled. So, we can leverage these to execute a custom piece of functionality when the cancellation token gets canceled. With this, we have implemented our pattern to cancel a non-cancellable task in C#. Now, let us use this pattern.

Example without using our Cancellation Token Pattern

```
using System;
using System.Threading.Tasks;
using System.Threading;
namespace AsynchronousProgramming
{
    class Program
    {
        static CancellationTokenSource cancellationTokenSource;
        static void Main(string[] args)
        {
            SomeMethod();
            Console.ReadKey();
        }
        public static async void SomeMethod()
        {
            cancellationTokenSource = new CancellationTokenSource();
            try
            {
                var result = await Task.Run(async () =>
                {
                    await Task.Delay(TimeSpan.FromSeconds(5));
                    Console.WriteLine("Operation was Successful");
                    return 7;
                }).WithCancellation(cancellationTokenSource.Token);
            }
            catch (Exception EX)
            {
                Console.WriteLine(EX.Message);
            }
            finally
            {
                cancellationTokenSource.Dispose();
                cancellationTokenSource = null;
            }
        }
    }
}
```

```

    }
}
}

```

Output: **Operation was Successful**

Example using our Cancellation Token Pattern

```

using System;
using System.Threading.Tasks;
using System.Threading;
namespace AsynchronousProgramming
{
    class Program
    {
        static CancellationTokenSource cancellationTokenSource;
        static void Main(string[] args)
        {
            SomeMethod();
            CancelToken();
            Console.ReadKey();
        }
        public static async void SomeMethod()
        {
            cancellationTokenSource = new CancellationTokenSource();
            try
            {
                var result = await Task.Run(async () =>
                {
                    await Task.Delay(TimeSpan.FromSeconds(5));
                    Console.WriteLine("Operation was Successful");
                    return 7;
                }).WithCancellation(cancellationTokenSource.Token);
            }
            catch (Exception EX)
            {
                Console.WriteLine(EX.Message);
            }
            finally
            {
                cancellationTokenSource.Dispose();
                cancellationTokenSource = null;
            }
        }
        public static void CancelToken()
        {
            cancellationTokenSource?.Cancel();
        }
    }
}

```

```
}
```

Output: The operation was canceled.

In the next article, I am going to discuss the [Asynchronous Streams in C#](#) with Examples. Here, in this article, I try to explain **Cancelling Non-Cancellable Tasks with TaskCompletionSource in C#** with Examples. I hope you enjoy this How to Cancel Non-Cancellable Tasks in C# with Examples article.